ARCHITECTURE — SCAFFOLD ONLY

Status: SCAFFOLD. Sections below contain prompts and TODO blocks. **You must fill these in yourselves.**

Per Tadashi CV non-negotiable rule on AI usage:

> Do not let AI own: project architecture, core algorithms, debugging first-pass thinking, data model design for important systems, performance reasoning, robotics or AI control logic you plan to claim as your strength.

This file is your responsibility. I have left the section headers, prompts, and reference patterns. The actual decisions, diagrams, schemas, and tradeoff reasoning must be written by Marv (Person 1 sections) and Ronin (Person 2 sections). Co-owners review.

When you fill in a TODO, delete the TODO marker and the surrounding prompt.

---

1. System overview

> **TODO (both):** One-paragraph plain-English description of what the system does, who uses it, and what makes it nontrivial. No marketing language. If you can't explain it without saying "AI-powered" or "intelligent," rewrite.

> **TODO (both):** Insert a high-level block diagram (Excalidraw → committed as PNG + `.excalidraw` source under `docs/diagrams/`). Show: client → service → agent core → inference + tool adapters, with telemetry instrumenting everything.

---

2. Component boundaries

2.1 Agent core (Marv)

> **TODO (Marv):** Describe the agent core's responsibilities in 3-5 bullets. What does it own? What does it explicitly *not* own?

> **TODO (Marv):** Document the agent loop as pseudocode (≤30 lines). Reference patterns to consider, not copy:
>
> - ReAct (reason-act-observe loop)
> - Plan-and-execute (LLM produces plan upfront, executor runs it)
> - Tool-use chat loop (OpenAI/Anthropic native tool-use API style)
>
> Pick one. Write down *why* you picked it. Name 2 alternatives you rejected and *why*.

2.2 Inference service (Marv)

> **TODO (Marv):** Describe the inference service's role and the request lifecycle. What gets batched? What's the cancellation story? Where does streaming begin and end?

> **TODO (Marv):** Document the backend-swapping interface. The same agent must run against OpenAI API, Anthropic API, or local MLX model with no agent-core code changes. Sketch the interface.

2.3 Telemetry (Marv)

> **TODO (Marv):** What gets logged at each layer? What's a trace? What's a span? Where does cost get computed? How do you correlate a user-visible failure with the underlying tool error?

2.4 Eval harness (Ronin)

> **TODO (Ronin):** Define the eval harness in 3-5 bullets. What's an "evaluation run"? What does the runner produce? How are baselines compared?

2.5 Model training pipeline (Ronin)

> **TODO (Ronin):** Describe the training pipeline stages: data extraction → format conversion → train/eval split → SFT run → checkpoint → registry. Where does leakage get prevented?

2.6 Tool adapters (Ronin)

> **TODO (Ronin):** List the tools, their auth model, their rate limits, their idempotency story. What does the adapter interface look like? How does the agent core discover available tools?

---

3. Core contracts (the most important section)

3.1 Tool-call contract

> **TODO (Marv):** Design the JSON schema for a tool call. Must support:
>
> - tool identifier
> - arguments (typed, validated)
> - expected state change (so eval can verify)
> - idempotency key
> - timeout / retry policy
>
> Commit the schema as `src/baymax/core/schemas/tool_call.schema.json`. Write it before any code uses it. The schema is the contract.

3.2 Scenario format

> **TODO (Ronin):** Design the JSON schema for an eval scenario. Must support:
>
> - user input (text, optional audio path)

- initial world state (fixtures for Notion/Calendar/etc.)
- expected outcome (state predicates, not literal tool-call sequences — multiple valid sequences must score correctly)
- ambiguity flag (if true, runner uses LLM-as-judge instead of exact match)
- tags (task type, difficulty, tools involved)

Commit as `src/baymax/eval/schemas/scenario.schema.json`.

3.3 Trace format

> **TODO (Marv):** Decide: OpenTelemetry spec, or custom? Document the decision. If custom, justify against OTel's overhead.

3.4 Model registry interface

> **TODO (Ronin with Marv review):** How is a model identified? Versioned? Loaded? How does the inference service ask the registry for "the best v1 fine-tuned model"?

---

4. Data flow

> **TODO (both):** Walk through one example end-to-end. User says "remind me about the article I just copied at 9am tomorrow." Trace it through:
>
> 1. ASR (if voice) or text input
> 2. Agent core receives intent
> 3. Tool selection (clipboard read → calendar create)
> 4. Tool execution
> 5. State change confirmation
> 6. Response to user
> 7. Telemetry written
>
> Be specific about where each component sits. Be specific about where failure can happen at each step.

---

5. State management

> **TODO (Marv):** Where does session state live? Per-process memory? Redis? SQLite? Justify the choice against the v1 requirements (single-user, multi-turn within session). Plan for v2 (multi-session memory).

---

6. Failure modes

> **TODO (Marv, with Ronin input from eval failures):** Enumerate failure modes and the agent's response. At minimum:

> - tool returns 4xx (user error in args)
> - tool returns 5xx (transient)
> - tool times out
> - model hallucinates a tool that doesn't exist
> - model produces malformed tool args
> - model refuses when it shouldn't
> - model proceeds when it should clarify
> - rate limit hit
> - auth expired
>
> For each: classification, response, telemetry, recovery action.

---

7. Security and privacy

> **TODO (Marv):** This system has access to your Notion/Gmail/Calendar. Document:
>
> - Where credentials live (env vars, OS keychain, never in source)
> - Whether traces redact sensitive content (yes, they must)
> - Whether the trained model could leak training-data PII into outputs (think about this before fine-tuning on your own emails)
> - What happens to scenario fixtures (don't commit real Notion content to the repo)

---

8. Performance budget

> **TODO (Marv):** Set the latency budget per stage. Suggested starting points to argue with:
>
> - Total user-perceived latency: P95 < 3000 ms for text, < 4000 ms for voice
> - ASR (if voice): P95 < 800 ms
> - Model first token: P95 < 1200 ms
> - Tool execution: P95 < 1000 ms
> - Streaming token rate: ≥ 30 tok/s on M-series Mac
>
> Justify each number. Numbers without justification are fake metrics (CV violation).

---

9. Decisions log (ADRs)

Every significant decision documented as a short ADR (Architecture Decision Record) at `docs/decisions/NNNN-title.md`:

- `0001-agent-loop-pattern.md` — Marv
- `0002-tool-call-schema.md` — Marv

- `0003-scenario-format.md` — Ronin
- `0004-base-model-choice.md` — Ronin
- `0005-inference-backend.md` — Marv
- `0006-trace-format.md` — Marv
- `0007-state-store.md` — Marv
- `0008-eval-judge-protocol.md` — Ronin

Each ADR ≤300 words: context, decision, alternatives considered, consequences. Use the Michael Nygard ADR template as reference.

---

10. Out of scope (anti-architecture)

> **TODO (both):** List what this system explicitly does not do, and why. This is as important as the in-scope list. See PROBLEM.md §"What BAYMAX v1 is NOT" — expand here.

How to use this scaffold

1. Each person fills in the sections they own. Co-owner reviews.
2. Sections 1, 4, 10 are filled jointly.
3. Section 3 (contracts) must be filled *before* Phase 1 code starts. Contracts before code.
4. Section 6 (failure modes) is updated continuously as eval surfaces real failures.
5. Section 9 (ADRs) grows organically — one ADR per significant decision.
6. When this scaffold has no more TODO blocks, delete this "How to use" section.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.